Numerical Methods

Lecture 2: Numerical Differentiation

1. 基础概念

1.1 Differential Equations (微分方程)

A **differential equation** is a mathematical equation that relates a function to its derivatives. Instead of just telling you the value of something, it describes how that value changes.

1.1.1 Forward Difference Method (前向差分法)

Formula:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Accuracy: First-order $O(h)$ (一阶精度，当h变小一倍，误差大约也变小一倍)

1.1.2 Central Difference Method (中心差分法)

Formula:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Accuracy: Second-order $O(h^2)$ (二阶精度，当h变小一倍，误差大约变小两倍)

1.1.3 Backward Difference Method (后向差分法)

Formula:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Accuracy: First-order $O(h)$ (一阶精度，当h变小一倍，误差大约也变小一倍)

1.2 Ordinary Differential Equations (ODEs) (常微分方程)

ODEs are differential equations involving functions of one independent variable and their derivatives. They can be classified by **order** (阶数) (highest derivative present).

1.2.1 First-Order ODEs (一阶常微分方程)

General form:

$$u'(t) = f(u(t), t)$$

$$u(t_0) = u_0$$

1.2.2 Second-Order ODEs (二阶常微分方程)

General form:

$$u''(t) = f(u(t), u'(t), t)$$

$$u(t_0) = u_0, \quad u'(t_0) = v_0$$

1.2.3 Higher-Order ODEs (高阶常微分方程)

General form:

$$u^{(n)}(t) = f(u(t), u'(t), u''(t), \ldots, u^{(n-1)}(t), t)$$

Note: Higher-order ODEs can always be converted to systems of first-order ODEs.

1.3 Taylor Series Method (泰勒级数法)

The Foundation: All numerical ODE methods are based on Taylor series expansion (泰勒级数展开):

$$u(t + \Delta t) = u(t) + \Delta t \cdot u'(t) + \frac{(\Delta t)^2}{2!} \cdot u''(t) + \frac{(\Delta t)^3}{3!} \cdot u'''(t) + \cdots$$

1.3.1 How it Works

1. Expand the solution using Taylor series
2. Truncate after desired number of terms
3. Use the ODE to substitute derivatives

1.3.2 Accuracy Based on Terms Kept

- Keep 2 terms: First-order $O(\Delta t)$ (一阶精度)
- Keep 3 terms: Second-order $O((\Delta t)^2)$ (二阶精度)
- Keep 4 terms: Third-order $O((\Delta t)^3)$ (三阶精度)

1.4 Forward Euler Method (Explicit Euler) (前向欧拉法/显式欧拉法)

1.4.1 Formula

$$u(t + \Delta t) = u(t) + \Delta t \times f(u(t), t)$$

1.4.2 Characteristics

- **Explicit (显式):** Easy to compute next value
- **First-order accurate (一阶精度):** $O(\Delta t)$
- **Can be unstable (可能不稳定)** for large time steps

1.5 Backward Euler Method (Implicit Euler) (后向欧拉法/隐式欧拉法)

1.5.1 Formula

$$u(t+\Delta t) = u(t) + \Delta t \times f(u(t+\Delta t), t+\Delta t)$$

1.5.2 Characteristics

- **Implicit (隐式):** Must solve equation for $u(t+\Delta t)$
- **First-order accurate (一阶精度):** $O(\Delta t)$
- **More stable (更稳定)** than Forward Euler

1.6 Heun's Method (Improved Euler/Predictor-Corrector) (Heun方法/改进欧拉法/预测-校正法)

1.6.1 Two-Step Process

Step 1 - Predictor (预测步):

$$\tilde{u}(t+\Delta t) = u(t) + \Delta t \times f(u(t), t)$$

Step 2 - Corrector (校正步):

$$u(t+\Delta t) = u(t) + \frac{\Delta t}{2} \times [f(u(t), t) + f(\tilde{u}(t+\Delta t), t+\Delta t)]$$

1.6.2 Characteristics

- **Explicit (显式):** No equation solving required
- **Second-order accurate (二阶精度):** $O((\Delta t)^2)$

1.7 Trapezoidal Method (Implicit) (梯形法/隐式梯形法)

1.7.1 Formula

$$u(t+\Delta t) = u(t) + \frac{\Delta t}{2} \times [f(u(t), t) + f(u(t+\Delta t), t+\Delta t)]$$

1.7.2 Characteristics

- **Implicit (隐式):** Must solve for $u(t+\Delta t)$
- **Second-order accurate (二阶精度):** $O((\Delta t)^2)$
- **Very stable (非常稳定)**

1.8 Errors (误差)

基本上Δx越小，order (阶数)越高，error (误差)越小

1.9 Composite methods（组合方法的优势）

1.9.1 Stronger Adaptability

Different parts of a function can behave differently (smooth vs. rapidly changing/singular), so different numerical integration methods may be suitable in different intervals. Hybrid methods pick the best method locally to improve accuracy.

For complex integrands with mixed components (e.g., polynomial + trigonometric), one method may struggle; hybrid methods can split the problem into parts, apply suitable methods to each part, and combine the results.

1.9.2 Balancing Precision and Efficiency

Hybrid methods can improve accuracy by using strengths of multiple approaches, such as doing partial analytical simplification first and then numerical integration for the remaining parts, without greatly increasing computation. Because methods differ in speed and cost, hybrid approaches choose faster methods for simple regions and higher-precision methods for difficult regions to optimize overall efficiency.

1.9.3 Enhanced Adaptive Capabilities

Hybrid methods often include adaptive integration (e.g., adaptive Simpson's), adjusting step size based on error estimates, and can combine other methods (like Gaussian quadrature) to handle locally complex regions. During computation, the algorithm can switch methods dynamically: use higher-precision methods where error is large, and use cheaper methods where the function varies slowly.

1.9.4 Reduced Error Propagation

Numerical integration errors can accumulate; hybrid methods reduce this by using different methods at different stages, such as starting with a high-precision method to limit early error growth. Hybrid methods can cross-check results using multiple methods; if results agree, they are reliable, and if not, the method can be adjusted or the interval subdivided.

1.9.5 Comprehensive Advantages

Hybrid methods are flexible because they can adapt strategies to many complex integration scenarios by combining analytical and numerical strengths. They are robust because if one method performs poorly in certain cases, other methods can still work, helping the overall integration succeed.

2 代码实现

2.1 Differentiation 求导

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

# Goal: Compare finite difference formulas for numerical derivative: forw
# Test function: f(t) = e^t, exact derivative: f'(t) = e^t evaluate at t

'''下面写的是前向差分方法，用来计算导数，核心思路是f'(t) ≈ [f(t+dt) − f(t)]/dt'''
# Forward difference:
# f'(t) ≈ [f(t+dt) − f(t)] / dt
# 1st-order accurate: O(dt)
```

```python
def forward_diff(f, t, dt):
    # this is my function answering question 1.1
    return (f(t + dt)- f(t)) / dt

'''下面写的是后向差分方法，用来数值计算导数，核心思路是f'(t) ≈ [f(t) - f(t-dt)]/dt
# Backward difference:
# f'(t) ≈ [f(t) - f(t-dt)] / dt
# 1st-order accurate: O(dt)
def backward_diff(f, t, dt):
    return (f(t) - f(t - dt)) / dt

'''下面写的是中心差分方法，用来数值计算导数，核心思路是f'(t) ≈ [f(t+dt) - f(t-dt)]
# Central difference:
# f'(t) ≈ [f(t+dt) - f(t-dt)] / (2*dt)
# 2nd-order accurate: O(dt^2)
def central_diff(f, t, dt):
    return (f(t + dt) - f(t - dt)) / (2 * dt)

'''下面定义的是测试函数f(t) = e^t，我们知道它的导数就是它本身'''
# Test function: f(t) = e^t, so exact derivative is also e^t
def fun(t):
    return np.exp(t)

'''设定测试点和精确解，在t=1处，e^t的导数就是e^1 = e ≈ 2.718'''
# Choose evaluation point t = 1, exact derivative = e^1
t = 1
exact = np.exp(t)

# Print exact derivative value
print('Exact derivative = ', exact)

# Print header (formatting for columns)
print('%20s%40s%40s' % ('Forward difference', 'Backward difference', 'Cen

'''定义几个列表来记录dt和对应的误差'''
# Store dt values and errors for convergence plot
fd_errors = [] # |forward_diff - exact|
bd_errors = [] # |backward_diff - exact|
cd_errors = [] # |central_diff - exact|
dt_all = [] # store dt sequence

'''然后就是循环测试不同的步长dt，从0.5开始，每次减半，总共测试10次'''
# Main loop: start dt=0.5, halve dt each time, run 10 tests
dt = 0.5
for i in range(10):

    '''用前向差分、后向差分和中央差分分别计算导数'''
    # Compute derivative approximations using three methods
    fd = forward_diff(fun, t, dt)
    bd = backward_diff(fun, t, dt)
    cd = central_diff(fun, t, dt)

    '''计算误差'''
    # Compute absolute errors
    err_fd = abs(fd - exact)
    err_bd = abs(bd - exact)
    err_cd = abs(cd - exact)

    '''打印结果和误差'''
    # Print results + errors
```

```
    print('%10g (error=%10.2g)         %10g (error=%10.2g)         %10g (
                (fd, err_fd, bd, err_bd, cd, err_cd))

    '''存储dt值和对应的误差，用于后面画图'''
    # Save dt and errors for plotting later
    dt_all.append(dt)
    fd_errors.append(err_fd)
    bd_errors.append(err_bd)
    cd_errors.append(err_cd)

    # Halve dt for next iteration
    dt = dt / 2

'''最后就是画图来展示误差和dt的关系，用对数坐标系可以清楚看到误差随步长的变化规律'''
# Convergence plot (log-log)
# If error ~ C * dt^p, then log(error) vs log(dt) is a straight line with
# Expect p≈1 for forward/backward, p≈2 for central.
fig = plt.figure(figsize=(7, 5))
ax1 = plt.subplot(111)

ax1.loglog(dt_all, fd_errors, 'b.-', label='Forward diff.')
ax1.loglog(dt_all, bd_errors, 'k.-', label='Backward diff.')
ax1.loglog(dt_all, cd_errors, 'r.-', label='Central diff.')

ax1.set_xlabel('$\Delta t$', fontsize=16)
ax1.set_ylabel('Error', fontsize=16)
ax1.set_title('Convergence plot', fontsize=16)
ax1.grid(True)
ax1.legend(loc='best', fontsize=14)

plt.show()
```

2.2 Differentiation using given formula 根据特定公式求导

有时候会给你一些新的公式，让你根据这个公式写ODE，比如2022年的考试，其实思路也是一样的

The first-order forward difference approximation to the derivative of f at location x is given by

$$f'(x) \approx \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

The second-order central difference approximation to the derivative of f at location x is given by

$$f'(x) \approx \frac{f(x+\Delta x) - f(x-\Delta x)}{2\Delta x}$$

The fourth-order central difference approximation to the derivative of f at location x is given by

$$f'(x) \approx \frac{-f(x+2\Delta x) + 8f(x+\Delta x) - 8f(x-\Delta x) + f(x-2\Delta x)}{12\Delta x}$$

For appropriate values of Δx (not too large, not too small) we expect all three of these to provide accurate approximations to the value of the derivative of f at x.

2.1 [10 marks]

Write functions to evaluate these three approximations given an arbitrary function f, location x and Δx value.

Check your implementations by evaluating the derivative of the function

$$f(x) = \sin(x)$$

at location $x = 1$.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

# Goal: Compare finite difference formulas for numerical derivative: forw
# Test function: f(x) = sin(x), exact derivative: f'(x) = cos(x) evaluate

'''下面写的是前向差分方法，用来计算导数，核心思路是f'(t) ≈ [f(t+dt) - f(t)]/dt'''
# Forward difference:
# f'(t) ≈ [f(t+dt) - f(t)] / dt
# 1st-order accurate: O(dt)
def forward_diff(f, x, dx):
    fx = f(x)
    fxph = f(x + dx)
    return (fxph - fx) / dx

'''这下面写的是二阶中心差分方法，核心思路是f'(x) ≈ [f(x+dx) - f(x-dx)]/(2*dx)'''
# 2nd-order central difference:
# f'(x) ≈ [f(x+dx) - f(x-dx)] / (2*dx)
# 2nd-order accurate: O(dx^2)
def central_diff2(f, x, dx):
    fxph = f(x + dx)
    fxmh = f(x - dx)
    return (fxph - fxmh) / (2 * dx)

'''这下面写的是四阶中心差分方法，核心思路是 f'(x) ≈ [-f(x+2dx) + 8f(x+dx) - 8f(x
# 4th-order central difference:
# f'(x) ≈ [-f(x+2dx) + 8f(x+dx) - 8f(x-dx) + f(x-2dx)] / (12*dx)
# 4th-order accurate: O(dx^4)
def central_diff4(f, x, dx):
    fxph = f(x + dx)
    fxp2h = f(x + 2*dx)
    fxmh = f(x - dx)
    fxm2h = f(x - 2*dx)
    return (-fxp2h + 8*fxph - 8*fxmh + fxm2h) / (12 * dx)

'''下面定义的是测试函数f(x) = sin(x)，我们知道它的导数是cos(x)'''
# Test function and exact derivative
def fun(x):
    return np.sin(x)

'''设定测试点和精确解，在x=1处，sin(x)的导数就是cos(1)'''
# Choose evaluation point t = 1, exact derivative = cos(1)
x = 1
```

```python
exact = np.cos(x)

# Print exact derivative value
print('Exact derivative = ', exact)

# Print header (formatting for columns)
print('%20s%40s%60s' % ('Forward difference', 'Central difference', '4th

'''定义几个列表来记录dx和三种方法对应的误差'''
# Store dt values and errors for convergence plot
fd_errors = [] # |forward_diff - exact|
cd2_errors = [] # |central_diff2 - exact|
cd4_errors = [] # |central_diff4 - exact|
dx_all = [] # store dt sequence

'''然后就是循环测试不同的步长dx，从0.5开始，每次减半，总共测试8次'''
# Main loop: start dx=0.5 and halve it each iteration
dx = 0.5
for i in range(8):

    '''用三种差分方法分别计算导数'''
    # Compute derivative approximations using three methods
    fd = forward_diff(fun, x, dx)
    cd2 = central_diff2(fun, x, dx)
    cd4 = central_diff4(fun, x, dx)

    '''存储dx值和对应的误差，用于后面画图'''
    # Store dx and corresponding absolute errors
    dx_all.append(dx)
    fd_errors.append(abs(fd - exact))
    cd2_errors.append(abs(cd2 - exact))
    cd4_errors.append(abs(cd4 - exact))

    # Halve dx for next iteration
    dx = dx / 2

'''这里打印相邻两次误差的比值，用来验证收敛阶数，理论上前向差分 p=1 -> 比值≈2；二阶中
# Print error ratios to verify convergence orders
# If error ~ C * dx^p, then error(dx)/error(dx/2) ~ 2^p.
# Expect ~2 (p=1), ~4 (p=2), ~16 (p=4).
for i in range(len(dx_all)-1):
    print(fd_errors[i]/fd_errors[i+1], cd2_errors[i]/cd2_errors[i+1], cd4

'''最后就是画图来展示误差和dx的关系，用对数坐标系可以清楚看到误差随步长的变化规律'''
# Convergence plot (log-log)
fig = plt.figure(figsize=(7, 5))
ax1 = plt.subplot(111)

ax1.loglog(dx_all, fd_errors, 'b.-', label='Forward diff.')
ax1.loglog(dx_all, cd2_errors, 'k.-', label='Central diff.2')
ax1.loglog(dx_all, cd4_errors, 'r.-', label='Central diff.4')

ax1.set_xlabel('$\\Delta x$', fontsize=16)
ax1.set_ylabel('Error', fontsize=16)
ax1.set_title('Convergence plot', fontsize=16)
ax1.grid(True)

'''导入用于绘制收敛阶数三角形标记的工具'''
# Fit straight lines in log-log space to estimate slopes (orders)
# log(error) ≈ intercept + slope * log(dx)
```

```
# slope ≈ order p
from mpltools import annotation

'''这是画趋近线的代码，实际上就替换数据就行，这里的start_fit参数是决定从哪一位开始画'
# which points to include in fitting (0 means use all)
start_fit = 0

'''分别为三种方法计算最佳拟合直线的斜率和截距'''
# Calculate the slope and intercept of the best-fitting straight line for
line_fit_fd = np.polyfit(np.log(dx_all[start_fit:]), np.log(fd_errors[sta
line_fit_cd2 = np.polyfit(np.log(dx_all[start_fit:]), np.log(cd2_errors[s
line_fit_cd4 = np.polyfit(np.log(dx_all[start_fit:]), np.log(cd4_errors[s

'''画出三条拟合直线，显示各自的收敛阶数（斜率）'''
# Draw three fitting lines and display their respective convergence order
# Plot fitted lines back in original scale: error ≈ exp(intercept) * dx^s
ax1.loglog(dx_all, np.exp(line_fit_fd[1]) * dx_all**(line_fit_fd[0]), 'k-
ax1.loglog(dx_all, np.exp(line_fit_cd2[1]) * dx_all**(line_fit_cd2[0]), '
ax1.loglog(dx_all, np.exp(line_fit_cd4[1]) * dx_all**(line_fit_cd4[0]), '

'''这是画三角形的代码，第一个变量是起始坐标位置，第二个变量定义斜率比率，分别标记1阶收
# Reference slope markers for p=1,2,4
annotation.slope_marker((2e-2, 5e-3), (1, 1), ax=ax1, size_frac=0.2, pad_
annotation.slope_marker((2e-2, 2e-5), (2, 1), ax=ax1, size_frac=0.2, pad_
annotation.slope_marker((2e-2, 1e-9), (4, 1), ax=ax1, size_frac=0.2, pad_

ax1.legend(loc='best', fontsize=14)

plt.show()
```

2.3 ODEs 常微分方程

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt

# Goal: Compare 4 time-stepping methods for solving the ODE:
# 1) FE: Forward Euler (1st order)
# 2) BE: Backward Euler (1st order)
# 3) ET: Euler-Trapezoid / Heun (2nd order)
# 4) IT: Implicit Trapezoidal / Crank-Nicolson (2nd order)
# Test ODE:  u'(t) = u with u(0)=1 on [0, t_max], exact solution: u(t) = 
# Then we run a convergence test by halving dt and measuring error at t=t

'''下面写的是前向欧拉方法(Forward Euler)，用来求解微分方程，核心思路是u_{n+1} = u_
# Forward Euler for u' = u:
# u_{n+1} = u_n + dt * u_n
def FE(u0,t0,t_max,dt):
    u=u0; t=t0; u_all=[u0]; t_all=[t0];
    while t<t_max:
        u = u + dt*u  # update using slope at (t_n, u_n)
        u_all.append(u)
        t = t + dt # advance time
        t_all.append(t)
    return(u_all,t_all)

"""下面写的是后向欧拉方法(Backward Euler)，用来求解微分方程，核心思路是u_{n+1} = u
# Backward Euler for u' = u:
# u_{n+1} = u_n + dt * u_{n+1}  ->  u_{n+1} = u_n / (1 - dt)
```

```python
def BE(u0,t0,t_max,dt):
    u=u0; t=t0; u_all=[u0]; t_all=[t0];
    while t<t_max:
        u = u/(1-dt)  # 后向欧拉的解析表达式
        u_all.append(u)
        t = t + dt
        t_all.append(t)
    return(u_all,t_all)

'''下面写的是Euler and Heun方法，核心思路是先用欧拉方法预测，再用梯形法校正'''
# Euler-Trapezoid / Heun method (predictor-corrector), 2nd order.
# Predictor (Euler): u_e = u_n + dt * u_n
# Corrector (trapezoid): u_{n+1} = u_n + (dt/2) * (f_n + f_e)
# Here f(u)=u, so: u_{n+1} = u_n + 0.5*dt*(u_n + u_e)
def ET(u0,t0,t_max,dt):
    u=u0; t=t0; u_all=[u0]; t_all=[t0];
    while t<t_max:
        ue = u + dt*u  # predictor: forward Euler guess
        u = u + 0.5*dt*(u + ue)  # corrector: average slope (trapezoid)
        u_all.append(u)
        t = t + dt
        t_all.append(t)
    return(u_all,t_all)

"""下面写的是隐式梯形法(Implicit Trapezoidal)，也叫Crank-Nicolson方法"""
# Implicit Trapezoidal / Crank-Nicolson, 2nd order.
# u_{n+1} = u_n + (dt/2)*(u_n + u_{n+1})
# u_{n+1} = u_n * (1 + dt/2) / (1 - dt/2) for u'=u
def IT(u0,t0,t_max,dt):
    u=u0; t=t0; u_all=[u0]; t_all=[t0];
    while t<t_max:
        u = u*(1+dt/2)/(1-dt/2)  # closed-form CN update for u'=u
        u_all.append(u)
        t = t + dt
        t_all.append(t)
    return(u_all,t_all)

'''计算error的公式'''

'''这里设置问题的初始参数：起始时间t0=0，初值u0=1，终止时间t_max=5'''
# Problem setup
t0 = 0.0
u0 = 1.0
t_max = 5.0

"""定义一个函数来计算四种数值方法在特定dt，和目标位置t_max下的误差"""
# Compute absolute error at t=t_max for each method.
def approx_error(dt, t_max):
    (u_all,t_all) = FE(u0,t0,t_max,dt)
    err1 = abs(u_all[-1] - np.exp(t_max))
    (u_all,t_all) = BE(u0,t0,t_max,dt)
    err2 = abs(u_all[-1] - np.exp(t_max))
    (u_all,t_all) = ET(u0,t0,t_max,dt)
    err3 = abs(u_all[-1] - np.exp(t_max))
    (u_all,t_all) = IT(u0,t0,t_max,dt)
    err4 = abs(u_all[-1] - np.exp(t_max))
    return err1, err2, err3, err4

'''定义几个列表来记录dt和四种方法对应的误差'''
# Store dt values and errors for convergence plot
```

```
error_FE = []
error_BE = []
error_ET = []
error_IT = []
dt_array = []

'''然后就是循环计算误差，dt从0.5开始，每次减半，直到dt小于5e-4'''
# Convergence loop: halve dt each time
dt = 0.5
while dt > 5e-4:
    dt_array.append(dt)
    err1, err2, err3, err4 = approx_error(dt, t_max)
    error_FE.append(err1)
    error_BE.append(err2)
    error_ET.append(err3)
    error_IT.append(err4)
    dt *= 0.5

'''画图，坐标轴是log，然后画三角形最后就是画图来展示误差和dt的关系，还是一样替换数据，
# Plot error vs dt in log-log scale
from mpltools import annotation

fig, ax1 = plt.subplots(1, 1, figsize=(10, 5))

# loglog plot: if error ~ C * dt^p, slope in log-log is p
ax1.loglog(dt_array, error_FE, 'b.', label='FE')
ax1.loglog(dt_array, error_BE, 'r.', label='BE')
ax1.loglog(dt_array, error_ET, 'g.', label='ET')
ax1.loglog(dt_array, error_IT, 'c.', label='IT')

ax1.set_xlabel('$\\Delta t$', fontsize=14)
ax1.set_ylabel('Error at $t=5$', fontsize=14)
ax1.set_title('Convergence test', fontsize=14)
ax1.grid(True)

# choose which points to use for fitting (ignore the first few coarse dt
start_fit = 4

'''分别为四种方法计算最佳拟合直线的斜率和截距'''
# Fit: log(error) = log(C) + p*log(dt) -> polyfit gives [p, log(C)]
line_fit_FE = np.polyfit(np.log(dt_array[start_fit:]), np.log(error_FE[st
line_fit_BE = np.polyfit(np.log(dt_array[start_fit:]), np.log(error_BE[st
line_fit_ET = np.polyfit(np.log(dt_array[start_fit:]), np.log(error_ET[st
line_fit_IT = np.polyfit(np.log(dt_array[start_fit:]), np.log(error_IT[st

'''画出四条拟合直线，显示各自的收敛阶数（斜率）'''
# Plot fitted lines back in original scale: error ≈ exp(intercept) * dt^s
ax1.loglog(dt_array, np.exp(line_fit_FE[1]) * dt_array**(line_fit_FE[0]),
ax1.loglog(dt_array, np.exp(line_fit_BE[1]) * dt_array**(line_fit_BE[0]),
ax1.loglog(dt_array, np.exp(line_fit_ET[1]) * dt_array**(line_fit_ET[0]),
ax1.loglog(dt_array, np.exp(line_fit_IT[1]) * dt_array**(line_fit_IT[0]),

'''这是画三角形的代码，第一个变量是起始坐标位置，第二个变量定义斜率比率；第一个三角形标
# Slope markers: show reference triangles for 1st-order and 2nd-order
annotation.slope_marker((1e-2, 9e-1), (1, 1), ax=ax1, size_frac=0.15, pad
annotation.slope_marker((1e-2, 2e-3), (2, 1), ax=ax1, size_frac=0.15, pad

ax1.legend(loc='best', fontsize=14)

plt.show()
```